

YANG-Push Contributions

This document is an excerpt from the PhD thesis authored by Alex Huang Feng. It provides background on YANG-Push and summarizes contributions that supported its industry adoption. For further details, see:

- Alex Huang Feng, “Service Provider Network Anomaly Detection: Standards, Architecture, and Algorithms for actual use by Network Operators”, PhD Thesis, INSA Lyon, 2026. Available: <http://e.pc.cd/VPKy6a1K>

This section presents our contributions to the development of the YANG-Push protocol [1]. We begin by describing the context and highlighting key limitations in the original specification that hindered adoption by network vendors and operators. We then detail the contributions made over four years of collaboration with Swisscom, NTT, and Huawei. These efforts focused not only on promoting industry adoption of YANG-Push, but also on ensuring its practical usability from an operator’s perspective, including its integration into broader data collection architectures.

A.1 What is YANG-Push?

YANG-Push is a network telemetry protocol standardized by the IETF in RFC 8641 [1]. It extends the capabilities of the YANG data modeling language [2] and the NETCONF/RESTCONF configuration protocols [3, 4] by enabling the subscription and streaming of data updates from network devices to external collectors. Instead of relying on periodic polling, YANG-Push allows network operators and monitoring systems to receive data updates proactively when values change or at specified intervals.

At its core, YANG-Push introduces a subscription-based model where clients can request specific data paths defined in the device’s YANG models. These models are instantiated following the principles of the Network Management Datastore Architecture (NMDA) [5], where operational and configuration state data are instantiated as in-memory data trees within the network device. YANG-Push enables continuous monitoring of these instantiated data trees by streaming updates to external collectors.

Once a subscription is established, the device streams data either periodically (according to a configured interval) or based on changes to the subscribed data nodes (on-change mode). This push-based telemetry mechanism was designed to fix the inefficiencies and limitations of polling-based telemetry protocols such as SNMP [6].

The messages produced by YANG-Push publishers are encoded using YANG-defined data encodings. Currently, three encodings are supported: XML [2], JSON [7], and CBOR [8]. Streaming data in these formats ensures that the messages are both machine-readable and semantically meaningful. Importantly, it also allows data collectors to validate each message

against the corresponding YANG model. Although this validation step may appear optional, it is in fact critical for operators. It helps identify software bugs, deviations from the specification, and inconsistencies in vendor implementations. This is especially important at the current stage of YANG-Push adoption, where device vendors are implementing the first releases according to the published specifications.

Although YANG-Push may appear to be a straightforward technology, its adoption has been limited by several factors. Among them are the slow pace of the IETF standardization process and the inherent complexity of the current specification, which led the industry to initially adopt alternative solutions. The following sections examine the key issues that have hindered the widespread adoption of YANG-Push. We describe these challenges in detail and present our contributions aimed at improving the protocol robustness and usability from an operator perspective.

A.1.1 Issues with YANG-Push

In this section, we explore the issues found when implementing and operating YANG-Push from the specified RFC 8641 [1].

A.1.1.1 Missing Specifications for a Valid Implementation

The YANG-Push specification started in 2015 and was finally published in September 2019, covering only the core protocol components [1, 9]. At the time of publication, no implementations were available. While the core documents defined the subscription model and device behavior, they left a critical gap: the specification of the transport protocols required to carry telemetry data in configured subscriptions. As a result, engineers aiming to deploy a standards-compliant YANG-Push solution were effectively blocked, as no standard mechanism existed for delivering the streamed data to collectors.

YANG-Push defines two types of subscriptions: *dynamic* and *configured*. In dynamic subscriptions, the client establishes the subscription at runtime, and the device sends notifications over the same session, typically a NETCONF session carried over SSH [10]. This model is fully supported by the core specification. In contrast, configured subscriptions decouple the subscription setup from the data transport. The destination and transport protocol are preconfigured on the network device, which then streams telemetry data independently to the designated collector. However, the lack of a standardized transport protocol for this model remains a fundamental obstacle to interoperable implementations.

Operationally, configured subscriptions are often preferred by service providers due to internal organizational constraints. Configuration and monitoring responsibilities are typically split across different teams, and the teams responsible for network monitoring may not have the permissions required to initiate NETCONF sessions, making dynamic subscriptions infeasible in practice. Configured subscriptions align better with these administrative boundaries by enabling push-based telemetry without requiring runtime interaction.

Despite its operational relevance, the YANG-Push specification does not define any standard transport protocol. To address this gap, we proposed a UDP-based transport for YANG-Push configured subscriptions. UDP offers a lightweight mechanism that can be implemented directly in router line cards, making it suitable for high-throughput scenarios and resource-constrained hardware. These design considerations are discussed in detail in Section A.2.1.

A.1.1.2 Complexity

A second factor that hindered the widespread adoption of YANG-Push is the complexity of its core components. The base specification includes advanced features that, while useful in specific

scenarios, significantly increase the implementation burden. One example is the support for event replay, which allows operators to receive notifications about events that occurred shortly before a subscription was established. Implementing this feature requires the router to cache event records in memory, increasing resource usage and implementation complexity. Another example is dampening, which allows routers to suppress the transmission of frequent, repetitive updates for rapidly changing data nodes. While this reduces the volume of telemetry traffic, it adds complexity to the logic for event generation and scheduling, making it more challenging for vendors to implement the full specification.

In recent discussions, service providers noted that many of the advanced features defined in the specification are not relevant to their daily operations. They emphasized the need for greater pragmatism from standard protocol developers and stressed that standards should be guided by real operational needs. In particular, they argued that implementations demonstrating concrete use cases should be presented before finalizing and publishing protocol specifications [11].

A.1.1.3 Slow Pace of IETF standardization process

One of the main factors that delayed the widespread adoption of YANG-Push was the slow pace of the IETF standardization process. Although YANG-Push provided a promising foundation for model-driven telemetry, the delay in completing the standard specification and the missing definition of critical components, such as the transport specifications, prevented early implementation and deployment. This prolonged timeline left a gap that the industry filled through alternative options, notably using non-standardized solutions.

It is in this context that gNMI (gRPC Network Management Interface) [12] was developed. Driven primarily by large network operators such as Google, gNMI was created as a pragmatic response to the lack of timely progress within the IETF. Rather than waiting for a complete and standardized solution, these operators chose to build their own framework using open-source technologies like gRPC [13]. This approach enabled rapid development, prototyping, and deployment, bypassing the slower consensus-driven path of formal standardization.

As a result, gNMI has gained significant traction, particularly in environments where speed of integration and toolchain compatibility are prioritized. Some network vendors have even built their entire network management systems around gNMI and gRPC. However, this fast-track development came with trade-offs. In the absence of standardized guidelines, each vendor remains free to define its own interpretation of the gNMI interface and architecture.

Service provider operating IP networks have raised concerns that while gNMI defines a common set of RPCs, its lack of normative constraints leads to inconsistent implementations. Despite claiming conformance with the same interface, each vendor tends to deliver a different variant. This fragmentation complicates multi-vendor deployments and forces operators to develop custom integrations, which ultimately increases operational complexity and cost.

While gNMI addressed the immediate need for a functional telemetry solution, operators continue to demand standards-based protocols that ensure interoperability. However, they also expect these protocols to be accompanied by reference implementations that demonstrate their feasibility and operational relevance before standardization is finalized. Without such validation, new protocols risk ending up like YANG-Push, formally specified but lacking implementations that prove their viability on existing hardware, which in turn delays adoption and industry trust.

A.1.1.4 Specification Gaps Enabling Integration with Downstream Systems

The messages used in YANG-Push are based on the legacy NETCONF Event Notification (RFC 5277) specification [14]. At the time, reusing this existing format was considered a practical choice, avoiding the need to redesign the entire messaging layer. However, this decision has revealed significant limitations over time. The message structure defined in RFC 5277 is

restricted to XML encoding and includes only a mandatory timestamp indicating when the message was generated. It lacks support for additional metadata, which is essential in modern telemetry architectures.

In large-scale deployments, telemetry data is collected through intermediate collectors that enrich the messages with metadata (Figure A.1). This typically includes tags identifying the source device and sequence numbers that allow operational teams to detect message loss and ensure end-to-end integrity. Once enriched, the messages are forwarded to message brokers such as Apache Kafka, which decouple producers and consumers and allow for scalable, asynchronous data processing.

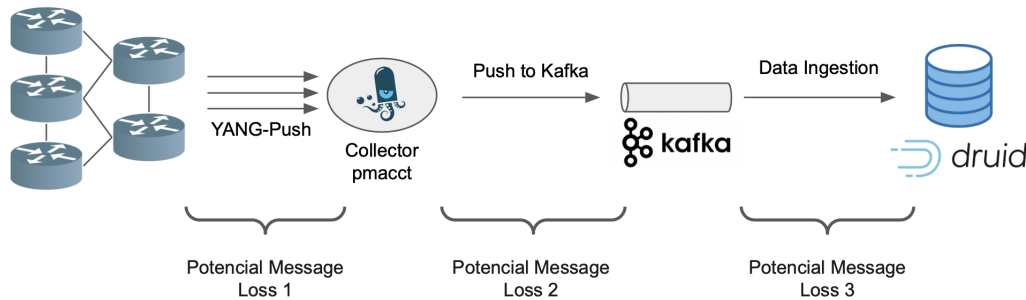


Figure A.1 – Typical Data Collection Chain.

In these architectures, data ingestion into backend systems, such as time series databases (TS-DBs), is handled by clients of the message broker that automate parsing and storage. Since the telemetry pipeline involves multiple components, message traceability becomes critical. When metrics from network devices fail to appear in the backend systems, operators must troubleshoot where in the pipeline the message was lost. Was the message dropped between the device and the collector? Between the collector and the message broker? Or was it mishandled by the client itself when ingesting into the database?

To support such diagnostic needs, operators require mechanisms that enable intermediate systems to trace message processing across the entire data pipeline. This, in turn, requires extending the message format to include additional metadata fields that can be added when the message is exported from the router. Our contributions address this gap by introducing a new extensible message header for telemetry notifications, as detailed in Section A.2.2.

A.2 Contributions to the Development of YANG-Push

In this section, we present our contributions to the development of YANG-Push. We aim at addressing key challenges faced by service providers in production networks. Building on the issues identified in the previous section, we introduce a new transport mechanism for configured subscriptions, propose an extensible message header and define the first set of critical metadata fields. These enhancements enable operators to trace telemetry messages throughout the data collection pipeline, improving reliability and operational visibility.

The following sections are structured as following. Section A.2.1 introduces UDP-Notif, a UDP-based transport protocol for YANG-Push configured subscriptions. This transport is designed for exporting periodic metrics from network devices, particularly in high-performance hardware such as router line cards. Section A.2.2 presents a new notification header that addresses operational challenges when integrating YANG-Push messages with downstream systems. This extensible header allows vendors to attach metadata that facilitates message tracking up to database ingestion. Finally, Section A.2.3 addresses limitations in the automation of YANG-Push data collection. We propose extensions to the YANG-Push capability discovery mecha-

nism, enabling collectors to automatically identify supported transports and notification header extensions.

A.2.1 UDP-Notif: UDP-based Transport for YANG-Push

YANG-Push targets two primary telemetry use cases: (i) periodic monitoring of device metrics (e.g., interface counters, CPU usage), and (ii) event-driven notifications triggered by changes to specific YANG data nodes (e.g., interface state transitions).

In the first case, where updates occur at regular intervals, occasional packet loss is tolerable, as the data will soon be refreshed. However, in the second case, reliability is critical. A single lost notification may cause the monitoring system to miss an important state change. For example, if a subscription is configured to notify changes in interface status, the loss of that notification may result in an inaccurate view of the network, potentially hiding a link failure or recovery event. In such scenarios, reliable delivery is essential to maintain accurate operational visibility.

We specified UDP-Notif [15] to address the first category of use cases, where reliability is not critical and lightweight transport is essential for efficient implementation in hardware line cards.

A.2.1.1 Why UDP?

In large-scale network telemetry deployments, relying on TCP-based transports introduces significant overhead. Maintaining a large number of active TCP sessions between collectors and network devices, especially on distributed architectures where telemetry data originates from individual line cards, creates scalability and resource management challenges. Each TCP connection requires memory, state maintenance, and protocol handling, which adds complexity and can become a bottleneck in high-density environments.

This problem has precedent. For example, although the IPFIX specification initially recommended using TCP as the transport, network vendors and operators widely adopted UDP instead. The motivation was clear: UDP stateless nature simplified implementation and minimized the load on forwarding plane components. The same rationale applies to YANG-Push, especially for periodic telemetry use cases where occasional packet loss is acceptable, and the primary goal is to export metrics that are regularly updated at fixed intervals.

A.2.1.2 Preference for Distributed Hardware Architectures

Modern routers are increasingly designed with distributed hardware architectures to maximize performance and scalability. Instead of relying on a centralized processor, critical functions, such as packet forwarding, are offloaded to dedicated line cards optimized for high-throughput operations. While this design enhances forwarding performance, it introduces challenges for telemetry processes. Collecting metrics from line cards at regular intervals can be inefficient and may interfere with the primary forwarding tasks, potentially affecting overall system performance.

To address these challenges, operators have advocated for extending the network telemetry approaches to support distributed architectures. The IETF draft on distributed notifications [16] follows this direction by enabling individual hardware components to generate and export telemetry data independently, while still aligning with the global data collection pipeline.

UDP-based transport aligns well with modern distributed router architectures. Its lightweight nature allows vendors to implement telemetry export functions directly on line cards, including caching the metrics, building the message and sending the notifications to collectors.

A.2.1.3 Open-source Implementations

UDP-Notif [15] is currently a proposed standard within the NETCONF Working Group. Major device vendors—including Huawei, Cisco, and 6WIND—have implemented it in their operating systems. Our team contributed by developing a reference implementation of the data collection suite. An open-source C library is available on GitHub [17], which has been integrated into the widely used data collection tool `pmacct` [18] and is actively collecting UDP-Notif messages from the Swisscom production network.

During the early development stages, we also created a Scapy-based implementation [19] to demonstrate the segmentation features of UDP-Notif. This project allowed us to validate our data collection library [17] and simulate real data streams while networking device implementations were still under development. The results were presented at the IETF 110 Hackathon [20].

In parallel, we explored automation of the data collection chain. Specifically, we analyzed the required metadata and RPC calls from the router to enable fully automated deployment of YANG-Push messages. These investigations led to the contributions presented in the following sections.

A.2.2 Notification Metadata

As discussed in Section A.1.1.4, collecting YANG-Push messages at scale involves passing them through collectors, message brokers, and databases. To support troubleshooting, network operators must be able to trace YANG-Push messages throughout the entire chain. When telemetry data is missing from the database, this visibility is essential to identify where messages may have been lost. To address these challenges, we present a new Notification header enabling vendors to add metadata helping operators track end-to-end the collected telemetry data.

A.2.2.1 Need for tracing telemetry messages to Downstream

The metrics and state data are located in the application layer of the message, which is subsequently encapsulated by each layer of the network stack to reach its destination. When messages fail to appear in downstream systems, network engineers must investigate whether the router was properly configured to send the notifications to the correct collector, or whether any system between the network device and the downstream system dropped the message. Engineers, similarly to monitoring the network, use counters to monitor these cases, such as the amount of forwarded messages or the amount of dropped messages. Each network layer can independently compute these metrics enabling operators to examine where in the network stack the message was dropped.

In typical data collection pipelines, the collector decapsulates the transport layer and forwards the notification content to the next component. Figure A.2 illustrates an example of a YANG-Push message transported over UDP-Notif. Upon receiving the message, the collector removes the UDP-Notif header, enriches the message with metadata, and forwards it to the message broker, adding a corresponding broker-specific header. At the end of the chain, a client consumes the message, strips off the message broker header, and stores the enriched message in the database.

To enable performance metrics to be computed across the entire data collection pipeline, YANG-Push messages must include both a unique identifier for the originating node and a sequence number. While the collector can use sequence numbers from the UDP-Notif header to detect message loss in the network, downstream components lack the necessary identifiers to perform similar assessments. Although the collector could generate its own sequence numbers and identifiers, this approach would lose the context of the original network node.

To address this, we advocate for embedding both the hostname and sequence numbers directly within the YANG-Push header, generated by the network device itself. Although this

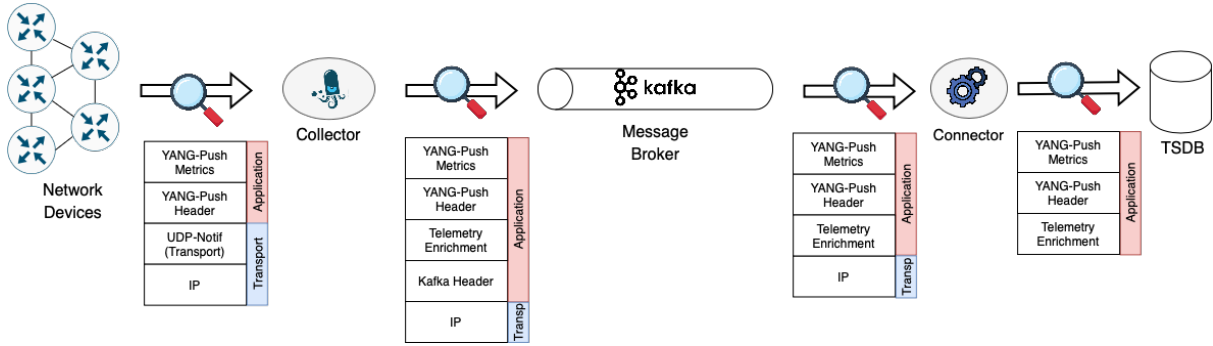


Figure A.2 – YANG-Push messages through each component of data collection.

appears to be a straightforward solution, the current specification, based on RFC 5277 [14], does not support header extensions, it only includes a timestamp indicating when the message was generated.

To overcome this limitation, we are defining at the NETCONF working group a new extensible notification header [21]. As part of this effort, we are introducing extensions to include the hostname and sequence numbers as a default enabled extensions. These additions will allow operators to track message loss at each stage of the pipeline, while preserving the context of the originating YANG-Push publisher.

This new header has been positively received by both operators and vendors. Major vendors, including Cisco and Huawei, have already committed to implementing it in their operating systems.

A.2.2.2 Fixing Time References

Correlating data across different network planes is essential for effective troubleshooting and monitoring. Accurate alignment of received events requires that each metric or network state produce a timestamp based on when the event actually occurred. However, because YANG-Push builds on top of RFC 5277 [14], the only timestamp available in the header reflects the time the message was generated, not the time of the underlying event. This limitation is particularly problematic when devices report state changes or metrics with a delay. In such cases, downstream systems, including databases, may mistakenly associate the event with the message generation time, leading to incorrect indexing and temporal misalignment.

To address this issue, we propose an extension to the YANG-Push header that introduces an observation-timestamp field. This timestamp explicitly indicates when the reported event took place, enabling accurate alignment of events during storage and analysis, and improving the fidelity of network telemetry data.

This extension was initially proposed as an independent Internet-Draft [22], but has since been integrated into the extensible notification header specification [21]. Major vendors have already added support for this timestamp in their YANG-Push implementations.

A.2.2.3 Robustness in Data Collection Tools

A message generated by a YANG-Push process adheres to a schema defined in YANG [2, 7]. The YANG model specifies the structure of the monitored Datastore [5], the semantics of each exported metric, and the data types of all fields. This enables automated parsing and interpretation of the data based on the predefined schema. Database vendors leverage these YANG definitions to generate and align their internal database schemas.

While matching a YANG-Push message to its corresponding set of YANG modules is a fundamental feature of NETCONF, it comes with important operational challenges. Each software

release supports only a specific version of this set, and upgrading the software may introduce non-backward compatible changes to the schemas. As a result, the structure or semantics of the exported messages may evolve. If the data collection pipeline is unaware of such a change, inconsistencies may arise. While components like the collector or Kafka may treat the message as generic JSON, the database—relying on strict schema definitions—may encounter messages that no longer conform to the expected schema, potentially breaking data ingestion.

This is a critical concern, as the data collection pipeline is a foundational component relied upon by network engineers for monitoring and troubleshooting. Any disruption in the pipeline can result in loss of visibility into the network state. To prevent this, it is essential to validate each message against the expected YANG schema before it reaches the database. Schema validation acts as a safeguard, allowing the system to detect and reject invalid messages early—before they cause ingestion failures.

To support this, we integrated YANG as a compatible schema type into the Confluent Schema Registry [23], which provides schema validation for Kafka messages. This integration allows systems that rely on the Schema Registry to automatically generate internal schemas use YANG as the schema definition language. Moreover, network operators can deploy the Schema Registry to validate incoming YANG-Push messages in real time, before they are consumed by Kafka clients. As a result, both database vendors and network operators can leverage existing validation infrastructure to ensure that only valid and compliant messages are processed, preserving the integrity and continuity of the data collection infrastructure.

The code of this integration has been made public on Github [23]. Example of clients producing and consuming YANG messages are also available [24].

A.2.3 YANG-Push Capabilities

Lastly, we investigated standards-based approaches to automate the instantiation of the data collection pipeline. A key question we addressed was how an orchestrator or operator can deploy a collector that automatically retrieves the relevant YANG schemas and configures UDP-Notif as the transport protocol, while accounting for the fact that, in production networks, not all devices may support UDP-Notif.

Section A.2.3.1 presents our extensions to the YANG Library, which enable dynamic discovery of the YANG modules supported by a network device, including modules that augment other YANG models. Section A.2.3.2 then introduces our extensions to the YANG module that defines YANG-Push capabilities. These enhancements allow for the dynamic discovery of supported transport protocols and their associated YANG encodings for YANG-Push publishers.

A.2.3.1 Automating data collection

When instantiating a collector, it must retrieve the full set of YANG modules associated with the YANG-Push messages it expects to receive. This set is necessary for validating messages through the Confluent Schema Registry and for enabling database vendors to generate their internal schemas accordingly.

Network devices typically support a large set of YANG modules. If the collector cannot determine which specific modules are relevant to the YANG-Push message, it must retrieve the entire set. This approach is inefficient: reports from operational environments indicate that retrieving the complete module set, including those unrelated to YANG-Push messages, can take up to 20 minutes [25]. Therefore, identifying the exact set of YANG modules associated with YANG-Push messages in advance significantly optimizes this process.

We propose using the YANG Library [26] to retrieve this relevant set. However, the standard YANG Library only reports modules directly used by the device, without including their recursive augmentation dependencies. As a result, if a YANG-Push message includes modules

that augment other modules, these dependencies are not reported, potentially missing YANG modules in the schema registration. This limitation is addressed by the augmentations proposed in [27], which extend the YANG Library to include augmentation relationships, ensuring that all required modules can be correctly discovered and retrieved.

A.2.3.2 Streamlining Transport Capabilities for YANG-Push

In production networks, devices often support different sets of transport protocols for YANG-Push, and each transport may support distinct encoding formats. For example, some devices may support lightweight, unreliable transports like UDP-Notif, while others may only offer reliable or proprietary options. Similarly, supported encodings —such as XML, JSON, or CBOR— can vary across devices or operating systems. This variability creates a challenge when deploying collectors in heterogeneous environments: each collector must be configured with a transport and encoding compatible with the target device.

Today, there is no standard way for collectors or orchestrators to dynamically discover which YANG-Push transports and encodings a device supports. Operators are often left to consult proprietary documentation or rely on vendor-specific tooling. This becomes a major operational burden in multivendor networks or when managing devices running different software versions.

To address this gap, we extend the system capabilities model [28] to include the set of supported YANG-Push transports, encodings, and encryption mechanisms [29]. These capabilities can be queried by collectors before any configuration is applied. This allows a collector to automatically select a compatible transport and encoding per device, or to prioritize specific transports based on performance, reliability, or operational preference.

This standards-based discovery mechanism enables automated deployment of data collection pipelines by eliminating manual configuration and reducing operational overhead. It is essential for ensuring the adaptability of telemetry systems in diverse, multivendor environments, common in medium and large service provider networks. Similar to other proposals, Huawei and Cisco already agreed on implementing these extensions.

A.3 Conclusion

All these standards-based extensions contribute to automating the collection of YANG-Push notifications. While they make YANG-Push suitable for large-scale deployment, they were driven by limitations in existing solutions. RFC 5277 [14] has seen limited adoption due to its rigid structure, lack of extensibility, and restricted transport options. Alternatives like gNMI [12], although gaining traction in some environments, do not align well with the operational models typically found in service provider networks. In these environments, configuration and telemetry are often handled by separate teams, yet gNMI assumes a tightly coupled model that blurs this distinction. Additionally, gNMI lacks standardization and often relies on proprietary interpretations of the architecture, increasing integration effort and operational complexity.

At the time of writing this thesis, all the proposed extensions have reached working group consensus and are under review by the Internet Engineering Steering Group (IESG) for publication.

These efforts are part of a broader architectural vision aimed at fully automating YANG-Push data collection in production networks, as outlined in [30]. Since the specification of YANG-Push, we have contributed to both the development of supporting standards and the implementation of key data collection components. The goal has been to enable operators and vendors to rely on a unified, interoperable solution, avoiding the need for proprietary alternatives.

Bibliography

- [1] A. Clemm and E. Voit, “Subscription to YANG Notifications for Datastore Updates,” Internet Requests for Comments, RFC Editor, RFC 8641, September 2019.
- [2] M. Björklund, “The YANG 1.1 Data Modeling Language,” RFC 7950, RFC Editor, Tech. Rep. 7950, Aug. 2016. [Online]. Available: <https://www.rfc-editor.org/info/rfc7950>
- [3] R. Enns, M. Björklund, A. Bierman, and J. Schönwälder, “Network Configuration Protocol (NETCONF),” RFC 6241, RFC Editor, Tech. Rep. 6241, Jun. 2011. [Online]. Available: <https://www.rfc-editor.org/info/rfc6241>
- [4] A. Bierman, M. Björklund, and K. Watsen, “RESTCONF Protocol,” RFC 8040, RFC Editor, Tech. Rep. 8040, Jan. 2017. [Online]. Available: <https://www.rfc-editor.org/info/rfc8040>
- [5] Martin Björklund and Jürgen Schönwälder and Philip A. Shafer and Kent Watsen and Robert Wilton, “Network Management Datastore Architecture (NMDA),” RFC 8342, RFC Editor, Tech. Rep. 8342, Mar. 2018. [Online]. Available: <https://www.rfc-editor.org/info/rfc8342>
- [6] D. Harrington, B. Wijnen, and R. Presuhn, “An Architecture for Describing Simple Network Management Protocol (SNMP) Management Frameworks,” RFC 3411, RFC Editor, Tech. Rep. 3411, Dec. 2002. [Online]. Available: <https://www.rfc-editor.org/info/rfc3411>
- [7] L. Lhotka, “JSON Encoding of Data Modeled with YANG,” RFC 7951, RFC Editor, Tech. Rep. 7951, Aug. 2016. [Online]. Available: <https://www.rfc-editor.org/info/rfc7951>
- [8] M. Veillette, I. Petrov, A. Pelov, C. Bormann, and M. Richardson, “Encoding of Data Modeled with YANG in the Concise Binary Object Representation (CBOR),” RFC 9254, RFC Editor, Tech. Rep. 9254, Jul. 2022. [Online]. Available: <https://www.rfc-editor.org/info/rfc9254>
- [9] E. Voit, A. Clemm, A. G. Prieto, E. Nilsen-Nygaard, and A. Tripathy, “Subscription to YANG Notifications,” RFC 8639, RFC Editor, Tech. Rep. 8639, Sep. 2019. [Online]. Available: <https://www.rfc-editor.org/info/rfc8639>
- [10] M. Cullen, “Using the NETCONF Protocol over Secure Shell (SSH),” RFC 6242, RFC Editor, Tech. Rep. 6242, Jun. 2011. [Online]. Available: <https://www.rfc-editor.org/info/rfc6242>
- [11] Thomas Graf, Holger Keller, Dan Voyer, Paolo Lucente, Benoit Claise, Rob Wilton, Alex Huang-Feng and Pierre Francois. (2024) Agile Incremental Driven Development for Network Management. [Accessed 2025-06-18]. [Online]. Available: <https://datatracker.ietf.org/doc/slides-interim-2024-nemopsws-02-sessa-5-agile-incremental-driven-development-for-network-management/>
- [12] Google, “gNxl Tools,” <https://github.com/google/gnxi>, accessed: 2024-11-25.

- [13] —, “gRPC,” <https://grpc.io>, accessed: 2024-11-25.
- [14] H. Trevino and S. Chisholm, “NETCONF Event Notifications,” RFC 5277, RFC Editor, Tech. Rep. 5277, Jul. 2008. [Online]. Available: <https://www.rfc-editor.org/info/rfc5277>
- [15] A. Huang Feng, P. Francois, T. Zhou, T. Graf, and P. Lucente, “UDP-based Transport for Configured Subscriptions,” Internet Engineering Task Force, Internet-Draft draft-ietf-netconf-udp-notif-21, May 2025, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-ietf-netconf-udp-notif/21/>
- [16] T. Zhou, G. Zheng, E. Voit, T. Graf, and P. Francois, “Subscription to Notifications in a Distributed Architecture,” Internet Engineering Task Force, Internet-Draft draft-ietf-netconf-distributed-notif-14, Jun. 2025, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-ietf-netconf-distributed-notif/14/>
- [17] A. Huang Feng and P. Francois. (2025) C-Collector for UDP-notif. [Accessed 2025-06-18]. [Online]. Available: <https://github.com/network-analytics/udp-notif-c-collector>
- [18] Paolo Lucente, “pmacct,” <http://pmacct.net>, 2003 - 2021.
- [19] A. Huang Feng and P. Francois. (2025) Scapy generator for UDP-notif. [Accessed 2025-06-18]. [Online]. Available: <https://github.com/network-analytics/udp-notif-scapy>
- [20] A. Huang Feng, P. Francois, T. Graf, and P. Lucente. (2021) Hackathon Results IETF 110. [Accessed 2025-06-18]. [Online]. Available: <https://github.com/IETF-Hackathon/ietf110-project-presentations/blob/main/ietf-110-hackathon-bmp-yang.pdf>
- [21] A. Huang Feng, P. Francois, T. Graf, and B. Claise, “Extensible YANG Model for YANG-Push Notifications,” Internet Engineering Task Force, Internet-Draft draft-ietf-netconf-notif-envelope-01, Mar. 2025, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-ietf-netconf-notif-envelope/01/>
- [22] T. Graf, B. Claise, and A. Huang Feng, “Support of Observation Timestamp in YANG-Push Notifications,” Internet Engineering Task Force, Internet-Draft draft-tgraf-netconf-yang-push-observation-time-03, Dec. 2024, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-tgraf-netconf-yang-push-observation-time/03/>
- [23] Huang Feng, Alex and Vivekananda Boudia and Ahmed Elhassany, “Integration of YANG to Confluent Schema Registry,” <https://github.com/network-analytics/yang-kafka-integration>, accessed: 2025-06-20.
- [24] Huang Feng, Alex and Vivekananda Boudia, “YANG Validation Samples,” <https://github.com/network-analytics/schema-registry-samples>, accessed: 2025-06-20.
- [25] Zhuoyao Lin and Benoit Claise. (2024) Hackathon Results IETF 120 on Augmented-by draft. [Accessed 2025-06-18]. [Online]. Available: https://github.com/IETF-Hackathon/ietf120-project-presentations/blob/main/Augmentedby_Addition_into_the_ietf_yang_library_ietf120_hackathon_slides.pdf
- [26] A. Bierman, M. Björklund, J. Schönwälder, K. Watsen, and R. Wilton, “YANG Library,” RFC 8525, RFC Editor, Tech. Rep. 8525, Mar. 2019. [Online]. Available: <https://www.rfc-editor.org/info/rfc8525>
- [27] Z. Lin, B. Claise, and I. D. Martinez-Casanueva, “Augmented-by Addition into the IETF-YANG-Library,” Internet Engineering Task Force, Internet-Draft draft-ietf-netconf-yang-library-augmentedby-05, May 2025, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-ietf-netconf-yang-library-augmentedby/05/>
- [28] B. Lengyel, A. Clemm, and B. Claise, “YANG Modules Describing Capabilities for Systems and Datastore Update Notifications,” RFC 9196, RFC Editor, Tech. Rep. 9196, Feb. 2022. [Online]. Available: <https://www.rfc-editor.org/info/rfc9196>

-
- [29] Q. Wu, Q. Ma, A. Huang Feng, and T. Graf, “YANG Notification Transport Capabilities,” Internet Engineering Task Force, Internet-Draft draft-ietf-netconf-yp-transport-capabilities-05, Jun. 2025, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-ietf-netconf-yp-transport-capabilities/05/>
 - [30] T. Graf and A. Elhassany, “An Architecture for YANG-Push to Message Broker Integration,” Internet Engineering Task Force, Internet-Draft draft-ietf-nmop-yang-message-broker-integration-07, Mar. 2025, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-ietf-nmop-yang-message-broker-integration/07/>